TypeScope.nvim — Project Requirements

A Neovim plugin that augments LSP signature/hover popups with rich, interactive data structure visualization for function parameters and return types.

Overview

When reading unfamiliar Python codebases, type annotations alone are often insufficient. `ServerConfig` tells you nothing about its shape. TypeScope bridges that gap by resolving types to their definitions and rendering their structure alongside the normal signature popup — with optional realistic example values generated locally.

Target Environment

- **Editor:** Neovim (0.10+)
- **Primary language:** Python
- **LSP:** basedpyright (required for type inference)
- **Parser:** nvim-treesitter (required for type definition extraction)
- **Optional backend:** Ollama (local LLM for example generation)

Core Features

1. Type Structure Resolution

Pipeline:

1. Cursor rests on a function call or definition → trigger via keymap or automatic on `CursorHold`
2. Request signature help via `vim.lsp.buf.signature_help()`
3. For each parameter and return type, fire `textDocument/definition` to locate the type declaration

4. Parse the type definition with TreeSitter to extract fields and their types
5. Recurse shallowly (configurable depth, default: 2 levels)

Supported type constructs (Python):

- `TypedDict`
- `dataclass` / `@dataclass`
- Pydantic `BaseModel`
- `NamedTuple`
- `Protocol`
- Built-in generics: `list[T]`, `dict[K, V]`, `tuple[...]`, `Optional[T]`, `T | None`, `Union[...]`

Fallback behavior:

- If LSP definition is unavailable, display raw annotation string
- If TreeSitter parsing fails, show type name only with a visual indicator

---

2. Visual Presentation

Layout — Augmented Float Window

TypeScope renders in a secondary float that anchors to the existing signature popup. The two floats are visually connected.

Sketch:

```
┌─ signature ──────────────────────────────────────────┐
│ create_server(config: ServerConfig, timeout: float)  │
│               ──────────────────                     │
└──────────────────────────────────────────────────────┘
┌─ typescope ──────────────────────────────────────────┐
│  ▾ config  ServerConfig                              │
│    │  host        str                                │
│    │  port        int                                │
│    │  debug       bool          = False              │
│    └─ timeout_ms  int | None    = None               │
│                                                      │
│  ▸ returns  Response            (press <CR> to expand)  │
```

└──┘

Visual Design Goals

- **Distinct identity:** Not a clone of existing popups (nvim-cmp, Telescope, lazy.nvim). Develop a visual spike (see §6) before committing to final chrome.
- **Accessibility:** High contrast between type names, field names, and annotations. Respect `background` option (dark/light). All decoration characters must have plain-text fallbacks for non-nerd-font environments.
- **Density balance:** Compact by default; never overwhelming. Long nested structures collapse by default.

Syntax Highlighting

- Field names, type names, default values, and structural chrome each get distinct highlight groups
- All highlight groups namespaced under `TypeScope*` for easy theme overrides
- Ships with sensible defaults that map to standard Neovim highlight groups as fallbacks (`@type` , `@variable` , `@keyword` , etc.)
- Supports Treesitter-injected highlighting inside the float for default value snippets

Tree Chrome Options (configurable)

Style	Characters
`unicode` (default)	`│ ├─ └─ ▾ ▸`
`ascii`	`| +- \- v >`
`minimal`	indentation only
`rounded`	`└─ ┌─ │`

---

3. Example Value Generation

Two modes, both cacheable by type-signature hash.

Mode A: Heuristic (default, zero latency)

Name + type pattern matching:

Field name pattern	Example value
`host` , `hostname` , `server`	`"localhost"`
`port`	`8080`
`timeout` , `ttl` , `interval`	`30.0` / `30`
`email`	`"user@example.com"`
`url` , `endpoint` , `uri`	`"https://example.com"`
`path` , `dir` , `file`	`"/tmp/example"`
`name`	`"example"`
`id` , `uuid`	`"a1b2c3d4"`
`bool` type	`True`
`int` (generic)	`42`
`float` (generic)	`3.14`
`str` (generic)	`"example"`
`None` / `Optional`	`None`

Mode B: LLM-Generated (opt-in, keymap triggered)

- Backend: **Ollama** HTTP API (`http://localhost:11434`)
- Recommended model: **Qwen2.5-Coder:3b** (fits CPU or ≤6GB VRAM)
- Prompt: structured type definition → realistic Python literal output
- Caching: keyed on canonical type structure string, persisted per session
- Latency indicator: spinner or progress in float title while generating
- Graceful degradation: falls back to heuristic if Ollama is unreachable

Rendered example:

```
  │ host        str          "localhost"
  │ port        int          8080
  │ debug       bool         False
  └─ timeout_ms  int | None   None
```

---

4. Interactivity

Keymap (default)	Action
`<CR>`	Expand / collapse node
`e`	Toggle example values
`E`	Trigger LLM example generation
`r`	Recurse deeper into selected type
`q` / `<Esc>`	Close TypeScope float
`?`	Show keymap help inline

All keymaps configurable. Float is navigable with `j` / `k`.

---

5. TUI Eye-Candy

- **Animated entrance:** float fades/slides in (optional, respects `vim.g.typescope_animations = false`)
- **LLM generation spinner:** Braille dot animation in float title bar while waiting for Ollama
- **Expand/collapse animation:** brief flicker-free height transition
- **Active parameter highlight:** synced with basedpyright's active parameter underline — the corresponding TypeScope entry pulses or highlights
- **Virtual text hints:** lightweight `▸ typescope` extmark on the function line indicating TypeScope data is available, without being intrusive

---

6. Visual Spike (Pre-Development Task)

Before implementing the final float chrome, build a throwaway prototype that explores

layout and visual identity options. Goals:

- Try 3–4 distinct visual styles (borders, colors, tree chrome, header layout)
- Test in both dark and light backgrounds
- Validate nerd-font vs plain-text fallback paths
- Gather feedback before committing to implementation

Spike output: a set of static Lua string renders or a simple test command that opens mock floats — no real LSP required.

Configuration Schema (draft)

```
require("typescope").setup({
  trigger = "hover",           -- "hover" | "manual"
  depth = 2,                   -- recursion depth for nested types
  show_examples = true,        -- show heuristic examples by default
  example_mode = "heuristic",  -- "heuristic" | "llm" | "none"
  ollama = {
    enabled = false,
    host = "localhost",
    port = 11434,
    model = "qwen2.5-coder:3b",
    timeout_ms = 8000,
  },
  ui = {
    style = "unicode",         -- "unicode" | "ascii" | "minimal" | "rounded"
    animations = true,
    max_width = 60,
    max_height = 20,
    border = "rounded",        -- any nvim float border value
    anchor = "signature",      -- "signature" | "cursor"
  },
  highlights = {
    -- override highlight groups here
  },
  keymaps = {
    expand = "<CR>",
    toggle_examples = "e",
    llm_generate = "E",
    recurse = "r",
    close = "q",
    help = "?",
```

```
  },
})
```

Out of Scope (v1)

- Non-Python languages (designed for future extension but not implemented)
- Remote Ollama endpoints (localhost only for now)
- Integration with nvim-cmp completion items
- Persistent cross-session type cache

Open Questions

1. Should TypeScope replace or augment the existing signature float? (Lean: augment — keep basedpyright's popup intact, TypeScope is additive)
2. Trigger behavior: always-on `CursorHold` vs explicit keymap? (Lean: keymap default, `CursorHold` opt-in to avoid performance concerns)
3. How to handle very wide nested types — horizontal scroll or truncation?
4. Should the visual spike be a standalone script or a `TypeScope spike` command inside the plugin itself?